Evaluación de modelos y ajuste de hiperparámetros

Sesión 12 · Módulo 3: Modelos de clasificación

Métricas avanzadas: MCC, kappa, log-loss · Desbalance de clases

GridSearchCV vs. RandomizedSearchCV · Calibración de probabilidades

Datos simulados · scikit-learn · imbalanced-learn

Plan de la sesión (≈100 min)

Qué veremos

- **A.** Por qué la exactitud (*accuracy*) nos puede engañar
- **B. Métricas avanzadas**: MCC, kappa de Cohen, log-loss
- **C. Desbalance de clases**: submuestreo, sobremuestreo, SMOTE y pesos de clase
- **D. Búsqueda de hiperparámetros**: `GridSearchCV` vs. `RandomizedSearchCV`
- **E. Calibración de modelos**: ¿qué tan confiables son las probabilidades?
- **F.** Síntesis: un flujo de evaluación completo

Una sola idea de fondo: evaluar bien un modelo no es calcular una sola cifra — es elegir la **métrica correcta**, ajustar sus **hiperparámetros** con rigor, y confiar en sus **probabilidades** tanto como en sus clases predichas.

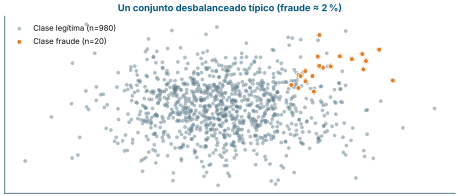
A1 · El problema: datos desbalanceados

Un caso común

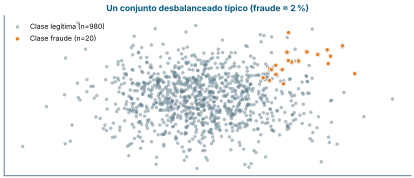
Detección de fraude, diagnóstico de enfermedades raras, detección de *spam* agresivo: la clase de interés es **rara** (1–5 % de los casos).

El modelo "perezoso"

Un clasificador que **siempre predice la clase mayoritaria** nunca detecta el fraude, pero obtiene una exactitud altísima.



A2 · El dataset de trabajo: detección de fraude



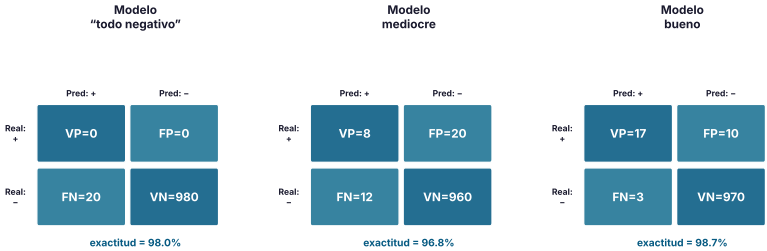
El dataset de ejemplo

- 1000 transacciones (simuladas)
- 980 legítimas (**98 %**)
- 20 fraudulentas (**2 %**)

Nota

Usaremos este mismo dataset en **todos** los ejemplos numéricos de la sesión.

A3 · Tres modelos de ejemplo



Los tres predicen sobre el **mismo** dataset (980 legítimas, 20 fraudes), pero distribuyen sus aciertos y errores de forma muy distinta — los usaremos a lo largo de la sesión para comparar métricas.

A4 · Las tres exactitudes se parecen demasiado

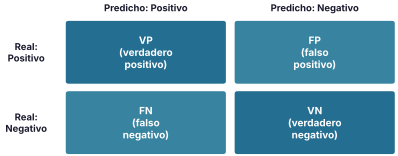
	Todo negativo	Mediocre	Bueno
Exactitud	98.0 %	96.8 %	98.7 %

El problema

Las tres exactitudes caen en un rango de apenas **2.7 puntos porcentuales** — a pesar de que el modelo "todo negativo" (A3) **nunca detecta un solo fraude** ($VP = 0$) y el modelo bueno detecta 17 de los 20.

La exactitud no logra separar tres comportamientos tan distintos. Necesitamos métricas que sí lo hagan — esa es la motivación de la sección B.

B1 · Matriz de confusión: precisión, exhaustividad y MCC



Precisión y exhaustividad (*recall*)

$$\text{Precisión} = \frac{VP}{VP + FP}$$

$$\text{Recall} = \frac{VP}{VP + FN}$$

Cada una mira **una sola** columna o fila de la matriz.

MCC: usa las cuatro celdas a la vez

$$MCC = \frac{VP \cdot VN - FP \cdot FN}{\sqrt{(VP + FP)(VP + FN)(VN + FP)(VN + FN)}}$$

$MCC \in [-1, 1]$: $+1$ predicción perfecta, 0 tan bueno como azar, -1 predicción perfectamente invertida.

B2 · MCC distingue lo que la exactitud no pudo

	Todo negativo	Mediocre	Bueno
Exactitud	98.0 %	96.8 %	98.7 %
MCC	0.00	0.32	0.73

```python
from sklearn.metrics import matthews_corrcoef

mcc = matthews_corrcoef(y_true, y_pred)
# funciona igual para 2 o mas clases
```

A diferencia de la exactitud (A4), que apenas varía 2.7 puntos entre los tres modelos, el MCC va de **0** (tan útil como el azar) a **0.73** (modelo bueno) — sí distingue un modelo útil de uno que no detecta un solo fraude.

B3 · Introduciendo kappa de Cohen (mismos tres modelos)

La idea

κ compara el acuerdo **observado** (p_o, = exactitud) contra el acuerdo **esperado por azar** (p_e, a partir de las proporciones marginales):

$$\kappa = \frac{p_o - p_e}{1 - p_e}$$

	T. neg.	Mediocre	Bueno
Exactitud	98.0 %	96.8 %	98.7 %
MCC	0.00	0.32	0.73
κ	0.00	0.32	0.72

Sobre estos mismos tres modelos (A3), κ se comporta casi igual que MCC: ambos corrigen por el acuerdo que ya se explicaría por azar bajo este desbalance (980 vs. 20) — en clasificación binaria, la diferencia entre las dos métricas suele ser pequeña.

B4 · Calculando kappa paso a paso (los tres modelos)

	Todo negativo	**Mediocre**	**Bueno**
p_o (= exactitud)	0.980	0.968	0.987
Marginales reales $(+, -)$	$(0.020, 0.980)$	$(0.020, 0.980)$	$(0.020, 0.980)$
Marginales predichas $(+, -)$	$(0.000, 1.000)$	$(0.028, 0.972)$	$(0.027, 0.973)$
Cálculo de p_e	$(.02)(0) + (.98)(1)$	$(.02)(.028)+(.98)(.972)$	$(.02)(.027)+(.98)(.973)$
p_e	$\mathbf{0.980}$	$\mathbf{0.953}$	$\mathbf{0.954}$
$\kappa = \dfrac{p_o - p_e}{1 - p_e}$	$\mathbf{0.000}$	$\mathbf{0.317}$	$\mathbf{0.717}$

p_o: la proporción **observada** de casos en los que la predicción coincide con la etiqueta real — es decir, la exactitud. p_e: la probabilidad de que **dos variables independientes** (la etiqueta real y la predicción) coincidan, si cada una mantiene su propia frecuencia marginal pero no tienen relación entre sí.

B5 · ¿Cuándo usar kappa de Cohen?

¿Las mismas situaciones que MCC?

En la práctica, cuando solo hay un "ground truth" y dos clases, MCC y κ casi siempre coinciden en qué modelo es mejor — usa κ sobre todo con más de dos clases o al comparar con anotadores humanos.

Pero κ tiene un uso que MCC no cubre igual de bien

Su dominio **original**: medir el **acuerdo entre dos anotadores** (dos personas, o una persona y un modelo) — estándar en anotación de datos, diagnóstico médico y estudios de confiabilidad inter-observador.

Escala habitual

< 0: peor que azar
0.01–0.20: leve
0.21–0.40: aceptable
0.41–0.60: moderado
0.61–0.80: sustancial
0.81–1.00: casi perfecto

B6 · Métricas “duras” vs. métricas “suaves”

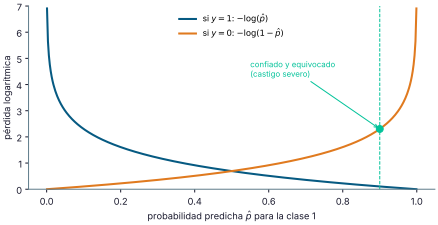
Exactitud, MCC y κ son métricas duras

Solo miran la **clase predicha final** (la que gana después de aplicar el umbral, típicamente 0.5). Si el modelo dice 51 % de fraude y el caso es fraude, cuenta igual que si hubiera dicho 99 %.

Log-loss es una métrica suave

Usa directamente la **probabilidad** que el modelo asignó, no la clase discretizada. Penaliza con fuerza creciente estar muy seguro y equivocarse.

B7 · Log-loss: penaliza la confianza equivocada



Definición (binaria)

$$\text{log-loss} = -\frac{1}{n}\sum_{i}\Big[y_i \log \hat{p}_i + (1-y_i)\log(1-\hat{p}_i)\Big]$$

A diferencia de exactitud/MCC/κ

No solo mira si la **clase** predicha es correcta — también la **probabilidad** asignada. Estar muy confiado y equivocarse se castiga mucho más que estar inseguro.

B8 · Un ejemplo numérico: calculando log-loss

4 predicciones

y_i	$\hat{p}_i$	término
1	0.9	$-\log(0.9) = 0.105$
0	0.2	$-\log(0.8) = 0.223$
1	0.6	$-\log(0.6) = 0.511$
0	0.3	$-\log(0.7) = 0.357$

Promediando los 4 términos

$$\text{log-loss} = \frac{0.105 + 0.223 + 0.511 + 0.357}{4}$$

Resultado

$$\text{log-loss} \approx \mathbf{0.299}$$

Fíjate en el caso 3 ($y = 1$, $\hat{p} = 0.6$): acertó la clase pero con poca confianza, y aun así aporta **más** pérdida (0.511) que el caso 1, que acertó con alta confianza (0.105) — log-loss premia estar en lo correcto *y* bien calibrado, no solo acertar la clase.

B9 · Log-loss en scikit-learn

```python
from sklearn.metrics import log_loss

proba = modelo.predict_proba(X_test)   # (n, n_clases)
loss = log_loss(y_test, proba)
# mientras mas bajo, mejor (0 = perfecto)
```

Log-loss exige `predict_proba`, no solo `predict`: es la métrica correcta cuando lo que de verdad se va a usar es la **probabilidad** (p. ej. decidir un umbral de alerta de fraude), no solo la etiqueta final — enlaza directo con la calibración de la sección E.

C1 · Tres formas de atacar el desbalance

Submuestreo

Undersampling. Elimina ejemplos de la clase **mayoritaria** hasta balancear.

Riesgo: perder información útil

Sobremuestreo

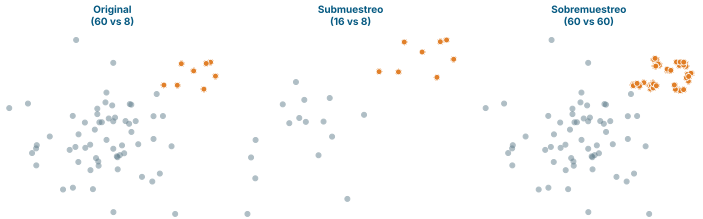
Oversampling. Duplica (o genera) ejemplos de la clase **minoritaria**.

Riesgo: sobreajuste a duplicados

Pesos de clase

No toca los datos: penaliza **más** los errores sobre la clase minoritaria durante el entrenamiento.

Más simple, no siempre suficiente



C2 · Submuestreo y sobremuestreo aleatorio en código

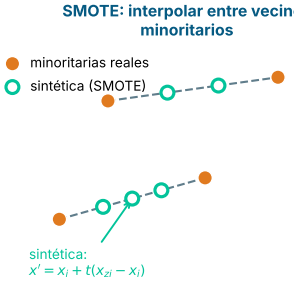
```python
from imblearn.under_sampling import RandomUnderSampler
from imblearn.over_sampling import RandomOverSampler

# submuestreo: iguala clases quitando mayoritaria
Xr, yr = RandomUnderSampler(random_state=0).fit_resample(Xtr, ytr)

# sobremuestreo: iguala clases duplicando minoritaria
Xr, yr = RandomOverSampler(random_state=0).fit_resample(Xtr, ytr)
```

Importante: el remuestreo se aplica **solo al conjunto de entrenamiento**, nunca al de prueba — de lo contrario evaluamos sobre datos artificiales y las métricas quedan infladas.

C3 · SMOTE: sobremuestreo sintético e inteligente



La idea

Synthetic Minority Oversampling Technique. Para cada punto minoritario, busca sus k vecinos minoritarios más cercanos e **interpola** nuevos puntos sintéticos entre ellos.

Ventaja sobre duplicar

Los puntos nuevos son **distintos** entre sí (no copias exactas), lo que reduce el riesgo de sobreajuste frente al sobremuestreo simple.

C4 · Cómo interpola SMOTE un punto sintético

Procedimiento

1. Toma un punto minoritario x_i y uno de sus k vecinos minoritarios más cercanos, x_{zi} (distancia euclidiana).
2. Elige al azar $t \sim \mathcal{U}(0,1)$.
3. Genera el punto sintético **sobre el segmento** que une ambos:

Fórmula de interpolación

$$x_{\text{nuevo}} = x_i + t\,(x_{zi} - x_i), \qquad t \sim \mathcal{U}(0,1)$$

El resultado es un punto en algún lugar **entre** x_i y x_{zi} (nunca fuera del segmento, pues $t \in [0,1]$) — por eso las muestras sintéticas quedan dentro de la región ya ocupada por la clase minoritaria, a diferencia de duplicar, que solo repite los puntos existentes.

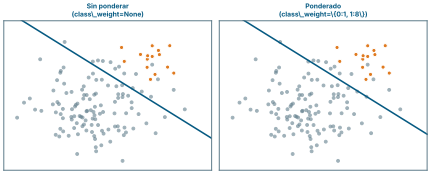
C5 · SMOTE en código

```
from imblearn.over_sampling import SMOTE

smote = SMOTE(k_neighbors=5, random_state=0)
Xr, yr = smote.fit_resample(Xtr, ytr)
# requiere variables numericas (o codificadas);
# SMOTENC existe para mezclas numerico/categoricas
```

SMOTE puede generar puntos sintéticos "ruidosos" si la clase minoritaria tiene *outliers*; variantes como `BorderlineSMOTE` o `SMOTEENN` (SMOTE + limpieza) suelen dar mejores resultados en la práctica.

C6 · Pesos de clase: la alternativa sin remuestrear




```python
from sklearn.ensemble import
    RandomForestClassifier

# calculado automaticamente (inverso a
    la frecuencia)
clf = RandomForestClassifier(
    class_weight="balanced",
        random_state=0)

# o pesos manuales por clase
clf = RandomForestClassifier(
    class_weight={0: 1, 1: 8},
        random_state=0)
```

Ponderar desplaza la **frontera de decisión** hacia la clase mayoritaria *menos* — sin inventar ni descartar un solo dato.
Disponible en casi todos los modelos de scikit-learn (regresión logística, SVM, árboles, bosques).

C7 · Ejemplo numérico: `class_weight="balanced"`

Fórmula

scikit-learn pondera cada clase en proporción **inversa** a su frecuencia:

$$w_j = \frac{n}{n_{\text{clases}} \cdot n_j}$$

Con nuestro dataset ($n = 1000$, $n_{\text{clases}} = 2$)

$$w_{\text{legítima}} = \frac{1000}{2 \cdot 980} \approx \mathbf{0.51} \qquad w_{\text{fraude}} = \frac{1000}{2 \cdot 20} = \mathbf{25.0}$$

Cada caso de fraude mal clasificado pesa **25/0.51 ≈ 49 veces más** que un caso legítimo mal clasificado — exactamente la razón de desbalance (980:20) del dataset. "balanced" calcula esto automáticamente, sin que lo definas a mano.

C8 · Ejemplo numérico: `class_weight={0:1, 1:8}`

Pesos manuales

Aquí **tú** eliges la razón entre pesos, en vez de usar la inversa exacta de la frecuencia:

$$w_{\text{legítima}} = 1, \qquad w_{\text{fraude}} = 8$$

Comparación de razones

	Razón fraude:legítima
`balanced` (automático)	$25.0/0.51 \approx 49{:}1$
`{0:1, 1:8}` (manual)	$8{:}1$

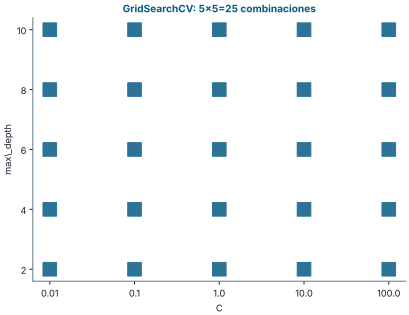
Con pesos manuales favoreces al fraude mucho **menos agresivamente** que "balanced" — útil cuando compensar la frecuencia exacta sobrecorrige y dispara demasiados falsos positivos; se ajusta por prueba y error según el costo real de cada tipo de error.

C9 · ¿Cuál técnica elegir?

	Ventaja	Cuándo usarla
Submuestreo / Sobremuestreo / SMOTE	rápido, simple / no se pierde información	dataset mayoritario enorme / clase minoritaria con pocos ejemplos
Pesos de clase	no altera los datos	primera opción, fácil de combinar con CV

En la práctica: empieza con `class_weight="balanced"` (más simple y menos riesgo de fuga de datos); si no basta, prueba SMOTE **dentro** de la validación cruzada con un `Pipeline` de `imblearn`.

D1 · GridSearchCV: búsqueda exhaustiva



La idea

Define una **malla** de valores para cada hiperparámetro y prueba **todas** las combinaciones con validación cruzada.

Costo

El número de combinaciones crece **multiplicativamente**: 5 valores $\times$ 5 valores $=$ 25 entrenamientos $\times k$ pliegues.

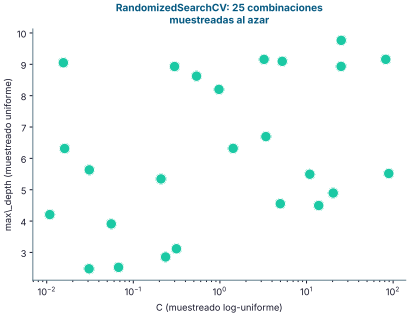
D2 · GridSearchCV en código

```python
from sklearn.model_selection import GridSearchCV

param_grid = {
    "C": [0.01, 0.1, 1, 10, 100],
    "max_depth": [2, 4, 6, 8, 10],
}
grid = GridSearchCV(
    estimator=modelo, param_grid=param_grid,
    scoring="matthews_corrcoef",   # OJO: elegir la metrica correcta (seccion B)
    cv=5, n_jobs=-1)
grid.fit(Xtr, ytr)
print(grid.best_params_, grid.best_score_)
```

El parámetro `scoring` conecta esta sección con la B: con clases desbalanceadas, **nunca** optimices por `accuracy` por defecto.

D3 · RandomizedSearchCV: muestreo aleatorio



La idea

En vez de una malla fija, **muestrea al azar** n combinaciones de distribuciones (uniforme, log-uniforme, ...) para cada hiperparámetro.

Ventaja

Con el **mismo presupuesto** de entrenamientos, explora un rango continuo — no solo los valores fijos de la malla.

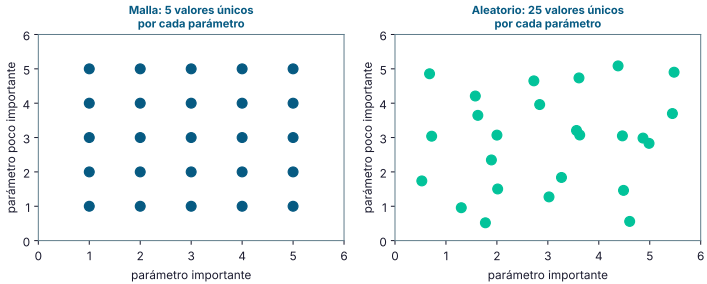
D4 · RandomizedSearchCV en código

```python
from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import loguniform, randint

param_dist = {
    "C": loguniform(1e-2, 1e2),
    "max_depth": randint(2, 11),
}
search = RandomizedSearchCV(
    estimator=modelo, param_distributions=param_dist,
    n_iter=25, scoring="matthews_corrcoef",
    cv=5, n_jobs=-1, random_state=0)
search.fit(Xtr, ytr)
```

`n_iter` controla el presupuesto directamente — a diferencia de `GridSearchCV`, donde el costo lo fija (indirectamente) el tamaño de la malla.

D5 · Por qué random gana cuando hay muchos hiperparámetros



Con el **mismo** número de evaluaciones, una malla repite el mismo valor del parámetro poco importante muchas veces; el muestreo aleatorio casi nunca repite un valor en **ningún** eje — cubre mejor al parámetro que sí importa (Bergstra & Bengio, 2012).

D6 · GridSearchCV vs. RandomizedSearchCV

	GridSearchCV	**RandomizedSearchCV**
Cobertura	exhaustiva sobre la malla	aleatoria, continua
Costo	crece multiplicativamente	lo fija `n_iter`
Pocos hiperparámetros	buena opción	también funciona
Muchos hiperparámetros	caro o inviable	**preferible**
Reproducibilidad	total	con `random_state`

Dos alternativas más eficientes que veremos a continuación: `HalvingGridSearchCV` (descarta candidatos pronto) y **Optuna** (búsqueda bayesiana adaptativa).

D7 · HalvingGridSearchCV: descarta candidatos pronto

Idea: *successive halving*

En cada ronda se mueven dos fracciones en sentido **opuesto**: la **fracción de datos** usada para evaluar empieza pequeña y **crece**; la **fracción de candidatos** que sobrevive a la siguiente ronda empieza en el 100 % de la malla y se **reduce** (se queda solo con los mejores). Así, hasta terminar con pocos candidatos evaluados con todos los datos.

```python
from sklearn.experimental import enable_halving_search_cv
from sklearn.model_selection import HalvingGridSearchCV

search = HalvingGridSearchCV(modelo, param_grid,
                             factor=3, resource="n_samples")
search.fit(Xtr, ytr)
```

No es búsqueda bayesiana: sigue evaluando una malla completa, pero **gasta menos tiempo** en combinaciones que ya se ven poco prometedoras con pocos datos — útil cuando entrenar con el dataset completo es costoso.

D8 · Optuna: búsqueda bayesiana adaptativa

Idea

Construye un modelo **probabilístico** de qué zonas del espacio de hiperparámetros dan buenos resultados, usando los **resultados de las pruebas anteriores** — y elige la siguiente combinación para explorar justo esas zonas prometedoras (en vez de malla fija o azar puro).

```python
import optuna

def objective(trial):
    c = trial.suggest_float("C", 1e-3, 1e2, log=True)
    modelo = LogisticRegression(C=c).fit(Xtr, ytr)
    return modelo.score(Xval, yval)

study = optuna.create_study(direction="maximize")
study.optimize(objective, n_trials=50)
```

Suele encontrar buenas combinaciones con **menos evaluaciones** que grid o random search, porque aprende de cada intento — a cambio, es secuencial por naturaleza y más difícil de paralelizar que una malla fija.

E1 · ¿Qué significa que un modelo esté “calibrado”?

Definición

Un modelo está **calibrado** si, para **cualquier** valor $p \in [0, 1]$, entre todos los casos a los que asigna probabilidad $\hat{p} = p$, aproximadamente una fracción p realmente son positivos.

Ejemplo: si $p = 0.8$, de esos casos cerca del 80 % deben ser positivos.

El problema

Muchos modelos (SVM, bosques, boosting) producen `predict_proba` **sin estar calibrados**: pueden ser sobreconfiados o subconfiados sin que accuracy lo note.

Importa siempre que la **probabilidad** se use para algo más que elegir la clase: fijar un umbral de alerta, combinar modelos, o reportar “nivel de riesgo” a un usuario.

E2 · ¿Por qué `predict_proba` no es una probabilidad real?

No nace como frecuencia observada

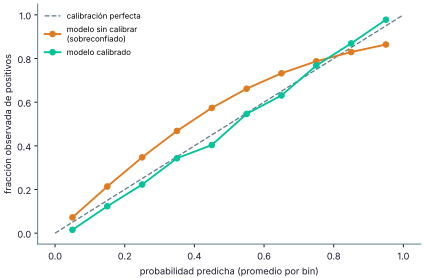
`predict_proba` es la salida de un mecanismo **interno** del modelo que tiene forma de probabilidad (vive en $[0, 1]$), pero que nunca fue obligado a coincidir con qué tan seguido acierta.

Según el modelo

- **Bosques**: fracción de árboles que votan por la clase — mide *acuerdo entre árboles*, no tasa de acierto.
- **SVM**: distancia al hiperplano, pasada por un sigmoide solo para que *parezca* probabilidad.
- **Boosting**: tiende a empujar las probabilidades hacia 0 o 1 en cada iteración correctiva.
- **Sobreajuste** (cualquier modelo): confianza alta en patrones memorizados que no generalizan.

Por eso hace falta un paso aparte de **calibración**: ajustar ese score heurístico para que sí coincida con la frecuencia real observada.

E3 · Diagrama de confiabilidad (*reliability diagram*)



Cómo se construye

Se agrupan las predicciones en *bins* por probabilidad predicha y se grafica, por bin, la probabilidad predicha promedio contra la fracción **real** de positivos.

La diagonal es la calibración perfecta. La curva naranja (sobreconfiada) se aleja de la diagonal — típico de SVM y de árboles/bosques sin calibrar.

E4 · Calibrando con CalibratedClassifierCV

```python
from sklearn.calibration import CalibratedClassifierCV, calibration_curve

# metodo "sigmoid" (Platt scaling): pocas muestras, forma ~sigmoide
# metodo "isotonic": mas muestras, mas flexible (no parametrico)
cal = CalibratedClassifierCV(
    estimator=modelo, method="isotonic", cv=5)
cal.fit(Xtr, ytr)
proba_cal = cal.predict_proba(Xte)[:, 1]

frac_pos, prob_media = calibration_curve(
    yte, proba_cal, n_bins=10)
```

`CalibratedClassifierCV` reentrena con validación cruzada: ajusta un mapeo ($\hat{p} \to p$ calibrada) **sin** tocar el modelo base ni ver el conjunto de prueba.

E5 · El “modelo base” y dos formas de usar `cv`

¿Qué es el “modelo base”?

El clasificador **ya entrenado** (el que produce las predicciones y el `predict_proba` sin calibrar) — `CalibratedClassifierCV` no toca sus predicciones de clase, solo le agrega una función que corrige las probabilidades.

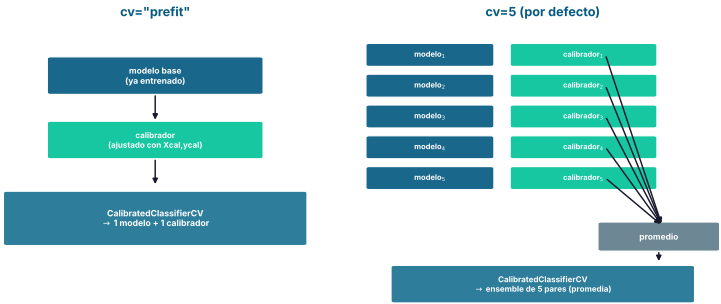
```
from sklearn.frozen import FrozenEstimator   # sklearn >= 1.6

# "congela" el modelo base (ya entrenado y fijo);
# solo se ajusta el calibrador con el set de calibracion
cal = CalibratedClassifierCV(FrozenEstimator(modelo_base))
cal.fit(Xcal, ycal)

# cv=5 (por defecto): reentrena el modelo base internamente
# en cada fold y promedia --- el resultado es un ensemble
cal = CalibratedClassifierCV(modelo_base, cv=5)
cal.fit(Xtr, ytr)
```

Con `cv=5`, las métricas del modelo calibrado sobre el conjunto de entrenamiento pueden verse distintas a las del modelo base original — no es un efecto de la calibración en sí, sino de que por dentro se reentrenó varias veces sobre menos datos cada vez. (En versiones de scikit-learn anteriores a la 1.6 se usaba `cv="prefit"` en vez de `FrozenEstimator`.)

E6 · Qué te queda al final: `prefit` vs. `cv=5`



En **ambos** casos lo que usas en producción es el objeto `CalibratedClassifierCV` completo, nunca el modelo base por separado — con `prefit` envuelve un solo par (modelo, calibrador); con `cv=5` envuelve 5 pares y **promedia** sus probabilidades calibradas en cada predicción.

E7 · Platt scaling vs. regresión isotónica

	Platt (sigmoid)	**Isotónica**
Forma	sigmoide (2 parámetros)	monótona, no paramétrica
Datos necesarios	funciona con pocos	necesita más datos
Riesgo	subajuste si la miscalibración no es sigmoide	sobreajuste con pocos datos

Regla práctica: con <1000 ejemplos de calibración usa `"sigmoid"`; con más datos, `"isotonic"` suele calibrar mejor.

E8 · ¿Cambian las métricas al calibrar?

Casi nunca cambia

El orden de los casos: Platt e isotónica son transformaciones **monótonas** del score. Por eso ROC-AUC y PR-AUC se mantienen prácticamente iguales.

Puede moverse en cualquier dirección

Accuracy, precisión, recall, MCC, κ: dependen de dónde cae el umbral (0.5) sobre las probabilidades, y calibrar puede desplazar casos de un lado al otro del umbral.

Suele mejorar

Log-loss y Brier score en validación/prueba — es justo lo que la calibración optimiza, si se hizo con datos separados del entrenamiento.

Entrenamiento es un caso aparte

Con el `cv` interno (E5), el modelo calibrado es un ensemble reentrenado — sus métricas sobre el propio set de entrenamiento pueden diferir del modelo base sin que eso sea "bueno" o "malo".

Riesgo real: un set de calibración muy pequeño puede sobreajustar al calibrador (sobre todo isotónica) y **empeorar** log-loss/Brier en vez de mejorarlos.

F1 · Un flujo de evaluación completo



Las cuatro piezas de hoy no son independientes: el **remuestreo** (C) cambia qué significa un buen `scoring` en la **búsqueda de hiperparámetros** (D); y tanto el remuestreo como el modelo elegido pueden **descalibrar** las probabilidades (E), que a su vez afectan directamente al log-loss (B).

Actividad de laboratorio (individual)

Flujo de evaluación completo: detección de fraude

Cuaderno de Jupyter con un conjunto sintético y desbalanceado de detección de fraude. Trabaja **de forma individual**.

1. Entrena un modelo base y reporta exactitud, MCC y kappa de Cohen; compara qué tan distinto es el panorama con cada métrica.
2. Calcula el log-loss usando `predict_proba` y compáralo contra el de un modelo "todo negativo".
3. Aplica **tres** estrategias de desbalance (`class_weight="balanced"`, `RandomOverSampler`, `SMOTE`) y compara su efecto en MCC.
4. Usa `GridSearchCV` y `RandomizedSearchCV` (mismo presupuesto de evaluaciones) para ajustar los hiperparámetros del mejor modelo de (3); compara tiempos y resultados.
5. Grafica la curva de calibración del modelo final antes y después de `CalibratedClassifierCV`.

Entregable: el cuaderno con la tabla comparativa de métricas y estrategias de desbalance, las curvas de calibración antes/después, y tu justificación de qué modelo (y qué umbral de decisión) recomendarías para producción.

Resumen

Las ideas centrales de hoy

- **MCC** y **kappa de Cohen** usan toda la matriz de confusión y corrigen por azar $\Rightarrow$ mucho más informativos que accuracy con clases desbalanceadas.
- **Log-loss** evalúa las **probabilidades**, no solo la clase predicha: castiga la confianza equivocada.
- Ante el **desbalance**: pesos de clase (más simple), sobremuestreo/SMOTE (más datos minoritarios), submuestreo (si hay sobra de datos mayoritarios).
- `GridSearchCV` es exhaustivo pero caro; `RandomizedSearchCV` cubre mejor el espacio con el mismo presupuesto cuando hay muchos hiperparámetros.
- Un modelo con buena exactitud puede estar **mal calibrado**: `CalibratedClassifierCV` (Platt o isotónica) corrige las probabilidades sin tocar el modelo base.

En la práctica: el `scoring` de la búsqueda de hiperparámetros debe ser la **misma** métrica avanzada que de verdad importa para el problema — nunca accuracy por omisión.



¿Preguntas?

¡Pasemos a la actividad de laboratorio: evaluación completa sobre detección de fraude!
